Skenario Kasus (Problem Statement)

Latar Belakang Kasus:

Anda bekerja sebagai **Associate Data Engineer / Data Wrangler** di sebuah platform retail omnichannel (*PT Nusantara Retail Mandiri*).

Head of Business Strategy (Data Owner) mengajukan kebutuhan informasi mendesak:

"Kami perlu menganalisis performa kampanye kupon diskon akhir bulan: Berapa total transaksi yang menggunakan kupon, bagaimana profil loyalitas pelanggan yang memakainya, serta apakah pesanan tersebut sudah terkirim oleh mitra logistik?"

Namun, data perusahaan tersimpan secara terpisah di tiga departemen:

1. **Divisi Kasir/POS:** Menyimpan log transaksi harian dalam bentuk berkas **CSV**.
2. **Divisi CRM / Membership:** Menyimpan data demografi dan level keanggotaan pelanggan di **Database Relasional (SQL/SQLite/PostgreSQL)**.
3. **Mitra Ekspedisi Logistik Pihak Ketiga:** Menyediakan status pengiriman paket secara real-time melalui **REST API (JSON)**.

📝 Instruksi & Langkah Pengerjaan

Bagian A: Analisis Konseptual & Tata Kelola Data (Bobot: 20%)

1. **Penerjemahan Kebutuhan Bisnis:**
   - Tuliskan analisis pemetaan dari *Kebutuhan Informasi Bisnis* menjadi *Kebutuhan Teknis Data* (tabelkan atribut, tipe data, dan sumber asalnya).
2. **Matriks Peran RACI:**
   - Buat tabel matriks RACI (*Responsible, Accountable, Consulted, Informed*) yang melibatkan:
     - *Head of Business Strategy* (Data Owner)
     - *Data Management Office / Data Steward*
     - *Anda / Tim Data Wrangling* (Data Custodian/Engineer)
     - *Business Intelligence Analyst* (Data User)

Bagian B: Komputasi Python & Data Ingestion Pipeline (Bobot: 50%)

Kerjakan di dalam Jupyter Notebook (`.ipynb`) dengan tahapan berikut:

1. **Ekstraksi Berkas File (CSV):**
   - Muat data transaksi penjualan dari file CSV.
   - Pastikan kolom tanggal di-*parse* ke tipe `datetime` dan kolom ID dipaksa bertipe `string` (`dtype={'order_id': str}`).
2. **Koneksi & Kueri Database (SQL):**
   - Sambungkan Python ke database lokal (gunakan SQLite atau PostgreSQL dengan `sqlalchemy`).

    - Ekstrak hanya pelanggan yang berstatus aktif (`WHERE is_active = 1`).
3. **Konsumsi REST API & Normalisasi JSON:**
    - Lakukan *request* GET ke endpoint API simulasi (menggunakan dummy REST API publik atau *mock JSON*).
    - Ratakan (*flatten*) struktur JSON bersarang menggunakan `pd.json_normalize()` agar siap digabung.
4. **Integrasi Data Multi-Sumber (*Merging*):**
    - Gabungkan ketiga dataframe tersebut menjadi satu dataframe master: `df_master_analisis`.
    - Gunakan tipe *join* yang tepat (`left` atau `inner`) dan sertakan alasan logis pemilihan join tersebut di sel *Markdown*.

Bagian C: Metadata, Kamus Data, & Validasi Aturan Bisnis (Bobot: 30%)

1. **Otomasi Kamus Data (*Data Dictionary*):**
    - Buat fungsi Python untuk menghasilkan ringkasan metadata dari `df_master_analisis`:
        - Nama Kolom
        - Tipe Data Sistem (`dtype`)
        - Jumlah Missing Value & Persentasenya
        - Jumlah Nilai Unik
        - Sampel Nilai
    - Simpan ringkasan ini menjadi file `kamus_data_hasil.csv`.
2. **Validasi Kualitas & Integritas (*Automated Assertions*):**
    - Tuliskan kode pengujian (`assert`):
        - Memastikan tidak ada duplikasi pada kolom kunci utama (*primary key*).
        - Memastikan kolom nilai transaksi (`total_amount`) tidak ada yang bernilai negatif.
        - Memastikan tidak ada nilai tanggal pesanan yang melebihi tanggal hari ini (*future date*).

Format Pengumpulan Luaran (*Deliverables*)

1. File arsip ZIP bernama `Tugas1_DW_NIM_NamaLengkap.zip` berisi:
    - File Notebook: `Tugas1_DW_NIM.ipynb` (sudah dalam keadaan seluruh *cell* berhasil di-*run* / *cleared outputs* tidak diperkenankan).
    - Dokumen Kamus Data: `kamus_data_hasil.csv`.
    - File dataset dummy penunjang (jika ada data lokal tambahan yang dibuat).
2. Tautan repositori GitHub pribadi yang memuat commit pengerjaan tugas (mendukung portofolio mahasiswa).

```python
# ================================================================================
# TUGAS MANDIRI DATA WRANGLING - PERTEMUAN 2
# Nama Mahasiswa :
# NIM            :
# ================================================================================

import pandas as pd
import numpy as np
import requests
from sqlalchemy import create_engine

# --- 1. EKSTRAKSI FILE CSV ---
print("[INFO] Mengimpor berkas transaksi POS...")
df_transaksi = pd.read_csv(
    "transaksi_pos.csv",
    dtype={"order_id": str, "customer_id": str},
    parse_dates=["order_date"]
)

# --- 2. EKSTRAKSI DATABASE SQL ---
print("[INFO] Menghubungkan ke database CRM...")
engine = create_engine("sqlite:///crm_loyalty.db")  # Sesuaikan kredensial Anda
query_crm = """
    SELECT customer_id, customer_name, membership_tier, city
    FROM customers
    WHERE is_active = 1
"""
df_crm = pd.read_sql(query_crm, con=engine)

# --- 3. KONSUMSI REST API (LOGISTIK) ---
print("[INFO] Menarik data pengiriman dari REST API...")
# Contoh menggunakan dummy/mock API endpoint
api_url = "https://api.mockfly.dev/mocks/dummy-logistik/deliveries"
response = requests.get(api_url, timeout=10)

if response.status_code == 200:
    data_logistik = response.json()
    # Flatten struktur nested JSON
    df_logistik = pd.json_normalize(
        data_logistik["results"],
        meta=["order_id", "tracking_number", "delivery_status"]
    )
else:
    print("[ERROR] Gagal memanggil API")

# --- 4. DATA INTEGRATION (MERGING) ---
print("[INFO] Melakukan integrasi data multi-sumber...")
df_master = df_transaksi.merge(df_crm, on="customer_id", how="inner")
df_master = df_master.merge(df_logistik, on="order_id", how="left")
```

```python
# --- 5. PEMBUATAN KAMUS DATA OTOMATIS ---
def generate_data_dictionary(df):
    dict_list = []
    for col in df.columns:
        dict_list.append({
            "Nama Kolom": col,
            "Tipe Data": str(df[col].dtype),
            "Missing Values": df[col].isnull().sum(),
            "Missing (%)": round((df[col].isnull().sum() / len(df)) * 100, 2),
            "Jumlah Unik": df[col].nunique(),
            "Contoh Nilai": str(df[col].dropna().iloc[0]) if not df[col].dropna().empty else "-"
        })
    return pd.DataFrame(dict_list)

kamus_df = generate_data_dictionary(df_master)
kamus_df.to_csv("kamus_data_hasil.csv", index=False)
print("[INFO] Kamus data berhasil disimpan.")
```

```python
# --- 6. ATURAN BISNIS & ASSERTIONS ---
assert df_master["order_id"].is_unique, "Fatal: Terdeteksi order_id duplikat!"
assert (df_master["total_amount"] >= 0).all(), "Fatal: Ada nilai transaksi negatif!"
print("[SUCCESS] Seluruh validasi data lulus!")
```

📊 Rubrik Penilaian Berbasis OBE (Skala 100)

Komponen & Bobot	Kriteria Sangat Baik (85 - 100)	Kriteria Cukup (70 - 84)	Kriteria Kurang (< 70)
Pemahaman Bisnis & Matriks RACI (20%)	Pemetaan kebutuhan data sangat presisi; matriks RACI membagi peran Owner, Steward, dan User secara realistis dan tepat.	Pemetaan ada namun kurang detail; pembagian peran RACI sedikit tumpang tindih.	Tidak ada pemetaan kebutuhan bisnis; matriks RACI keliru atau tidak dibuat.
Ingestion Pipeline Multi-Sumber (40%)	Berhasil menarik data dari CSV, SQL, dan API secara sempurna; penanganan *nested JSON* tepat; penggabungan tabel (*join*)	Berhasil menarik data namun penanganan JSON atau penentuan jenis join (*left/inner*) kurang tepat.	Gagal menarik data dari salah satu sumber; terdapat error eksekusi kode.

Skenario Kasus (Problem Statement)

Latar Belakang Kasus:

Anda bekerja sebagai **Associate Data Engineer / Data Wrangler** di sebuah platform retail omnichannel (*PT Nusantara Retail Mandiri*).

Head of Business Strategy (Data Owner) mengajukan kebutuhan informasi mendesak:

"Kami perlu menganalisis performa kampanye kupon diskon akhir bulan: Berapa total transaksi yang menggunakan kupon, bagaimana profil loyalitas pelanggan yang memakainya, serta apakah pesanan tersebut sudah terkirim oleh mitra logistik?"

Namun, data perusahaan tersimpan secara terpisah di tiga departemen:

1. **Divisi Kasir/POS:** Menyimpan log transaksi harian dalam bentuk berkas **CSV**.
2. **Divisi CRM / Membership:** Menyimpan data demografi dan level keanggotaan pelanggan di **Database Relasional (SQL/SQLite/PostgreSQL)**.
3. **Mitra Ekspedisi Logistik Pihak Ketiga:** Menyediakan status pengiriman paket secara real-time melalui **REST API (JSON)**.

📝 Instruksi & Langkah Pengerjaan

Bagian A: Analisis Konseptual & Tata Kelola Data (Bobot: 20%)

1. **Penerjemahan Kebutuhan Bisnis:**
   - Tuliskan analisis pemetaan dari *Kebutuhan Informasi Bisnis* menjadi *Kebutuhan Teknis Data* (tabelkan atribut, tipe data, dan sumber asalnya).
2. **Matriks Peran RACI:**
   - Buat tabel matriks RACI (*Responsible, Accountable, Consulted, Informed*) yang melibatkan:
     - *Head of Business Strategy* (Data Owner)
     - *Data Management Office / Data Steward*
     - *Anda / Tim Data Wrangling* (Data Custodian/Engineer)
     - *Business Intelligence Analyst* (Data User)

Bagian B: Komputasi Python & Data Ingestion Pipeline (Bobot: 50%)

Kerjakan di dalam Jupyter Notebook (`.ipynb`) dengan tahapan berikut:

1. **Ekstraksi Berkas File (CSV):**
   - Muat data transaksi penjualan dari file CSV.
   - Pastikan kolom tanggal di-*parse* ke tipe `datetime` dan kolom ID dipaksa bertipe `string` (`dtype={'order_id': str}`).
2. **Koneksi & Kueri Database (SQL):**
   - Sambungkan Python ke database lokal (gunakan SQLite atau PostgreSQL dengan `sqlalchemy`).

    - Ekstrak hanya pelanggan yang berstatus aktif (`WHERE is_active = 1`).
3. **Konsumsi REST API & Normalisasi JSON:**
    - Lakukan *request* GET ke endpoint API simulasi (menggunakan dummy REST API publik atau *mock JSON*).
    - Ratakan (*flatten*) struktur JSON bersarang menggunakan `pd.json_normalize()` agar siap digabung.
4. **Integrasi Data Multi-Sumber (*Merging*):**
    - Gabungkan ketiga dataframe tersebut menjadi satu dataframe master: `df_master_analisis`.
    - Gunakan tipe *join* yang tepat (`left` atau `inner`) dan sertakan alasan logis pemilihan join tersebut di sel *Markdown*.

Bagian C: Metadata, Kamus Data, & Validasi Aturan Bisnis (Bobot: 30%)

1. **Otomasi Kamus Data (*Data Dictionary*):**
    - Buat fungsi Python untuk menghasilkan ringkasan metadata dari `df_master_analisis`:
        - Nama Kolom
        - Tipe Data Sistem (`dtype`)
        - Jumlah Missing Value & Persentasenya
        - Jumlah Nilai Unik
        - Sampel Nilai
    - Simpan ringkasan ini menjadi file `kamus_data_hasil.csv`.
2. **Validasi Kualitas & Integritas (*Automated Assertions*):**
    - Tuliskan kode pengujian (`assert`):
        - Memastikan tidak ada duplikasi pada kolom kunci utama (*primary key*).
        - Memastikan kolom nilai transaksi (`total_amount`) tidak ada yang bernilai negatif.
        - Memastikan tidak ada nilai tanggal pesanan yang melebihi tanggal hari ini (*future date*).

Format Pengumpulan Luaran (*Deliverables*)

1. File arsip ZIP bernama `Tugas1_DW_NIM_NamaLengkap.zip` berisi:
    - File Notebook: `Tugas1_DW_NIM.ipynb` (sudah dalam keadaan seluruh *cell* berhasil di-*run* / *cleared outputs* tidak diperkenankan).
    - Dokumen Kamus Data: `kamus_data_hasil.csv`.
    - File dataset dummy penunjang (jika ada data lokal tambahan yang dibuat).
2. Tautan repositori GitHub pribadi yang memuat commit pengerjaan tugas (mendukung portofolio mahasiswa).

```python
# ============================================================================
# TUGAS MANDIRI DATA WRANGLING - PERTEMUAN 2
# Nama Mahasiswa :
# NIM            :
# ============================================================================

import pandas as pd
import numpy as np
import requests
from sqlalchemy import create_engine

# --- 1. EKSTRAKSI FILE CSV ---
print("[INFO] Mengimpor berkas transaksi POS...")
df_transaksi = pd.read_csv(
    "transaksi_pos.csv",
    dtype={"order_id": str, "customer_id": str},
    parse_dates=["order_date"]
)

# --- 2. EKSTRAKSI DATABASE SQL ---
print("[INFO] Menghubungkan ke database CRM...")
engine = create_engine("sqlite:///crm_loyalty.db")  # Sesuaikan kredensial Anda
query_crm = """
    SELECT customer_id, customer_name, membership_tier, city
    FROM customers
    WHERE is_active = 1
"""
df_crm = pd.read_sql(query_crm, con=engine)

# --- 3. KONSUMSI REST API (LOGISTIK) ---
print("[INFO] Menarik data pengiriman dari REST API...")
# Contoh menggunakan dummy/mock API endpoint
api_url = "https://api.mockfly.dev/mocks/dummy-logistik/deliveries"
response = requests.get(api_url, timeout=10)

if response.status_code == 200:
    data_logistik = response.json()
    # Flatten struktur nested JSON
    df_logistik = pd.json_normalize(
        data_logistik["results"],
        meta=["order_id", "tracking_number", "delivery_status"]
    )
else:
    print("[ERROR] Gagal memanggil API")

# --- 4. DATA INTEGRATION (MERGING) ---
print("[INFO] Melakukan integrasi data multi-sumber...")
df_master = df_transaksi.merge(df_crm, on="customer_id", how="inner")
df_master = df_master.merge(df_logistik, on="order_id", how="left")
```

```python
# --- 5. PEMBUATAN KAMUS DATA OTOMATIS ---
def generate_data_dictionary(df):
    dict_list = []
    for col in df.columns:
        dict_list.append({
            "Nama Kolom": col,
            "Tipe Data": str(df[col].dtype),
            "Missing Values": df[col].isnull().sum(),
            "Missing (%)": round((df[col].isnull().sum() / len(df)) * 100, 2),
            "Jumlah Unik": df[col].nunique(),
            "Contoh Nilai": str(df[col].dropna().iloc[0]) if not df[col].dropna().empty else "-"
        })
    return pd.DataFrame(dict_list)

kamus_df = generate_data_dictionary(df_master)
kamus_df.to_csv("kamus_data_hasil.csv", index=False)
print("[INFO] Kamus data berhasil disimpan.")
```

```python
# --- 6. ATURAN BISNIS & ASSERTIONS ---
assert df_master["order_id"].is_unique, "Fatal: Terdeteksi order_id duplikat!"
assert (df_master["total_amount"] >= 0).all(), "Fatal: Ada nilai transaksi negatif!"
print("[SUCCESS] Seluruh validasi data lulus!")
```

📊 Rubrik Penilaian Berbasis OBE (Skala 100)

Komponen & Bobot	Kriteria Sangat Baik (85 - 100)	Kriteria Cukup (70 - 84)	Kriteria Kurang (< 70)
Pemahaman Bisnis & Matriks RACI (20%)	Pemetaan kebutuhan data sangat presisi; matriks RACI membagi peran Owner, Steward, dan User secara realistis dan tepat.	Pemetaan ada namun kurang detail; pembagian peran RACI sedikit tumpang tindih.	Tidak ada pemetaan kebutuhan bisnis; matriks RACI keliru atau tidak dibuat.
Ingestion Pipeline Multi-	Berhasil menarik data dari	Berhasil menarik data	Gagal menarik data dari